

Denis Kibichiy

BSCCS/2024/55349

BSC Computer Science

Unit Code: BIT3208 Advanced Web Design and Development

Michael Nyoro

May/August 2025/2026

Project Details

Project Title: Runi Empire Grooming Barbershop Management System

Selected Technology: JavaScript, PHP, MySQL, XAMPP

GitHub Link:

https://github.com/deniskibichiy/BIT3208_Advanced_Web_Design_and_Development#

Table of Contents

1. Week 1: Environment setup and database connection.....	4
1.1. Figure 1: XAMPP Installation and Setup with MySQL in Linux.....	4
1.2. Figure 2: Apache + MySQL Running and Operational.....	4
1.3. Figure 3: Index.php successfully rendered.....	5
1.4. Figure 4: Database connection successful.....	5
1.5. Figure 5: Week1 database.....	6
Week 1 reflection:.....	6
2. Week 2: User Interface Planning and System Design.....	7
2.1. Figure 2.1: Homepage User Interface Mockup.....	7
2.2. Figure 2.2: Login Page User Interface Mockup.....	8
2.3. Figure 2.3: Admin Dashboard User Interface Mockup.....	9
2.4. Figure 2.4: Homepage Figma prototype.....	10
2.5. Figure 2.5: Login page Figma prototype.....	11
2.6. Figure 2.6: Dashboard Figma prototype.....	12
2.7. Figure 2.7: Color Scheme.....	13
2.8. Figure 2.8: Navigation Structure.....	13
Reflection week 2:.....	14
3. Week 3: JavaScript and PHP Basics.....	14
3.1. Figure 1: Invalid Login Validation.....	14
3.2. Fig 2: Valid login input accepted.....	15
3.3. Fig 3: DOM manipulation interaction.....	15
3.5. Figure 5: Week 3 database connection success evidence.....	16
Week 3 Reflection:.....	16
4. WEEK 4: Server-Side Components and Backend Development Foundations.....	17
4.1. Figure 1: Login page initial state.....	17
4.2. Figure 2: Failed Login Attempt.....	17
4.3. Figure 3: Successful login attempt - Redirected to dashboard.....	17
4.4. Figure 5: Folder Structure.....	18
Week 4 Reflection:.....	18
5. Week 5: Database Components and CRUD Operations.....	19
5.1. Figure 5.1: Databases Screenshot.....	19
5.2. Figure 5.2: Created Tables within Database and defined their schema.....	19
5.3. Figure 5.3: CRUD Operations on database.....	20
5.4. Figure 5.4: Database Connection.....	20
5.5. Figure 5.5: Successful login through the database.....	21
Week 5 Reflection:.....	21
6. Week 6: Database Integration and CRUD Operations.....	22

6.1. Database Creation (Already done in week 5 but now normalized).....	22
6.2. Entity Relationship Diagram.....	23
6.3. User Registration Form on Submit submits to the Users table.....	23
6.4. Update and Delete Records via the terminal evidence.....	24
Week 6 reflection:.....	25
7. Week 7: User Authentication and Session Management.....	25
7.1. Figure 7.1: users table with different roles included.....	25
7.2. Figure 7.2: Role assignments done successfully.....	26
7.3. Figure 7.3: Normal User Registration.....	26
7.4. Figure 7.4: Normal User saved with role successfully.....	26
7.5. Figure 7.5: Password hashing for security purposes.....	27
7.6. Figure 7.6: Super Admin Dashboard with super admin privileges extends 2 admin privileges (creating users and bookings).....	27
7.6.1. Super admin can edit admins.....	27
7.6.2. Super admin can create services.....	28
7.7. Admin dashboard showing admin privileges that can also be done by super admin.....	28
7.7.1. Admin can create users (admin + super admin).....	28
7.7.2. Admin can create bookings that can also be done by the super admin.....	29
7.7.3. Admin can't edit fellow admins or super admin.....	29
7.8. Normal user profile with limited privileges: edit profile, create booking and cancel booking.....	29
7.8.1. Normal user can edit profile.....	30
7.8.2. Normal user can view their bookings and add or cancel.....	30
7.8.3. A normal user can add a booking.....	30
7.9. Week 7 reflection.....	30
Week 8: Responsive Design.....	31
7.10. Figure 8.1: Home page desktop view.....	31
7.11. Figure 8.2: Home page mobile view.....	32
7.12. Figure 8.3: Navigation Desktop view.....	32
7.13. Figure 8.4: Navigation Mobile view.....	33
7.14. Figure 8.5: Responsive tables using horizontal scroll.....	33
7.15. Week 8 reflection.....	33

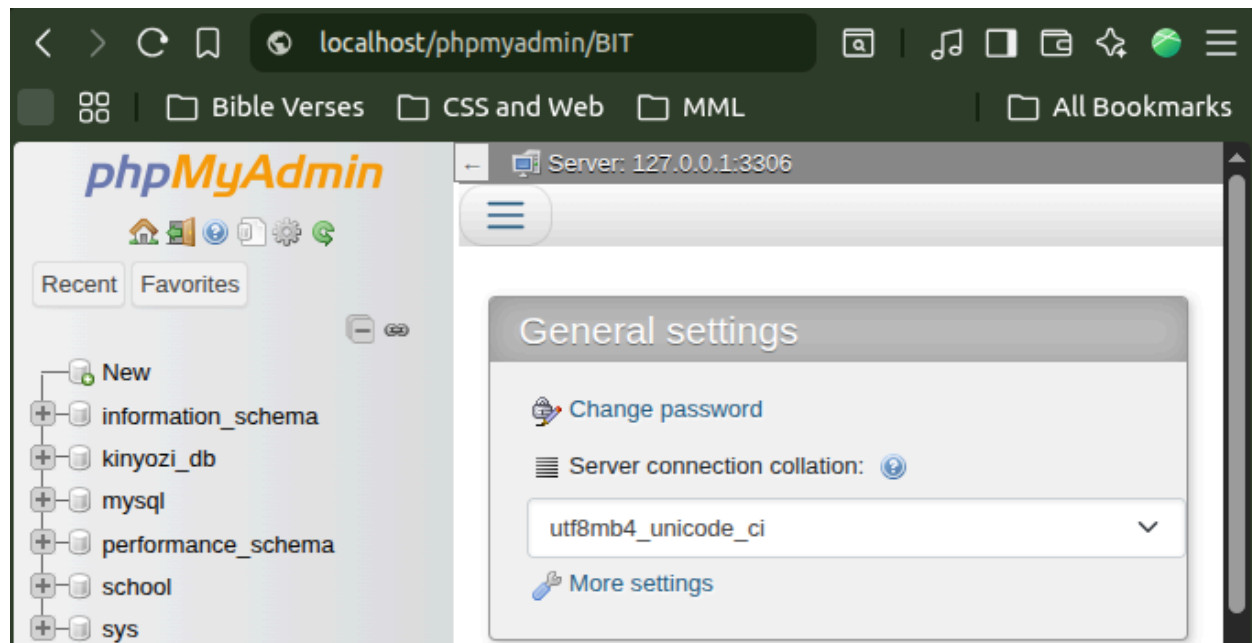
1. Week 1: Environment setup and database connection

1.1. Figure 1: XAMPP Installation and Setup with MySQL in Linux

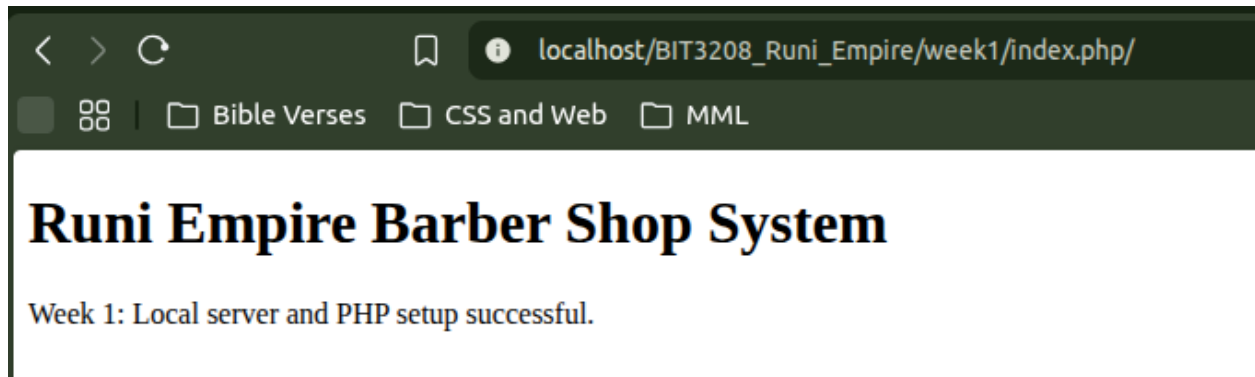
```
denis@denis-HP-ProBook-450-G7:~$ sudo /opt/lampp/xampp status
Version: XAMPP for Linux 8.2.12-0
Apache is running.
MySQL is not running.
ProFTPD is not running.
denis@denis-HP-ProBook-450-G7:~$ sudo systemctl status mysql
● mysql.service - MySQL Community Server
   Loaded: loaded (/lib/systemd/system/mysql.service; enabled; vendor p
   Active: active (running) since Thu 2026-05-28 13:18:55 EAT; 39min ago
   Main PID: 10633 (mysqld)
   Status: "Server is operational"
   Tasks: 39 (limit: 18907)
   Memory: 434.2M
   CPU: 14.863s
   CGroup: /system.slice/mysql.service
           └─10633 /usr/sbin/mysqld

May 28 13:18:53 denis-HP-ProBook-450-G7 systemd[1]: Starting MySQL Commu
May 28 13:18:55 denis-HP-ProBook-450-G7 systemd[1]: Started MySQL Commu
denis@denis-HP-ProBook-450-G7:~$ ss -tulnp | grep 3306
tcp  LISTEN 0      70          127.0.0.1:3306  0.0.0.0:*
tcp  LISTEN 0      151         127.0.0.1:3306  0.0.0.0:*
denis@denis-HP-ProBook-450-G7:~$
```

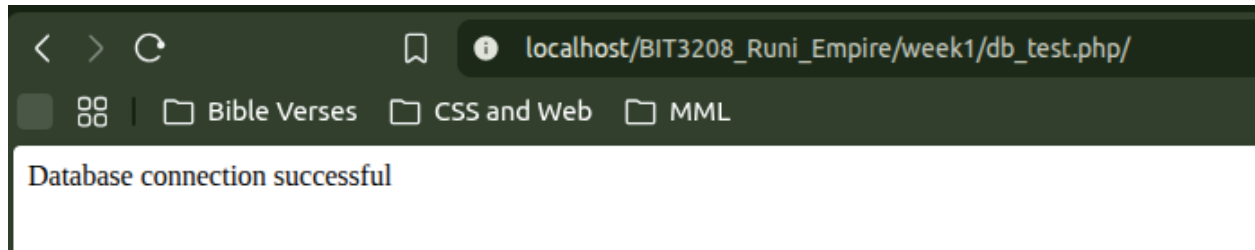
1.2. Figure 2: Apache + MySQL Running and Operational



1.3. Figure 3: Index.php successfully rendered



1.4. Figure 4: Database connection successful



1.5.Figure 5: Week1 database

```
mysql> USE week1db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_week1db |
+-----+
| users              |
+-----+
1 row in set (0.01 sec)

mysql> DESCRIBE users
-> ;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int  | NO   | PRI | NULL    | auto_increment |
| name  | varchar(100) | YES |     | NULL    |               |
| phone_number | varchar(20) | YES |     | NULL    |               |
| created_at | timestamp | YES |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

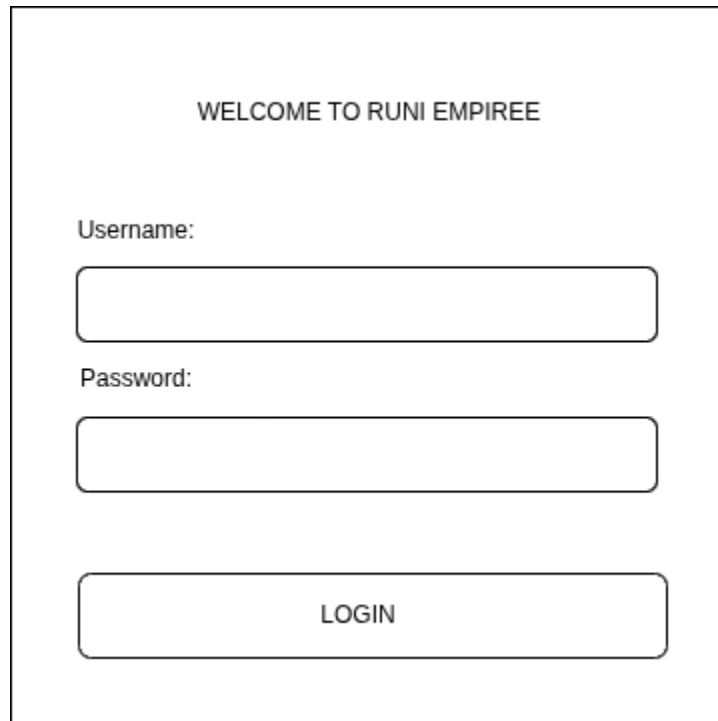
Week 1 reflection:

During week one, the focus was on setting up the environment to facilitate practical activities which are part of the Unit requirements. A key challenge I experienced is that the XAMPP MySQL couldn't load successfully due to port clashes with the system MySQL. To solve this, I had to configure Apache to use system MySQL instead of the one provided by PHPMyAdmin, and the process was successful.

2. Week 2: User Interface Planning and System Design

2.1. Figure 2.1: Homepage User Interface Mockup



2.2. Figure 2.2: Login Page User Interface Mockup

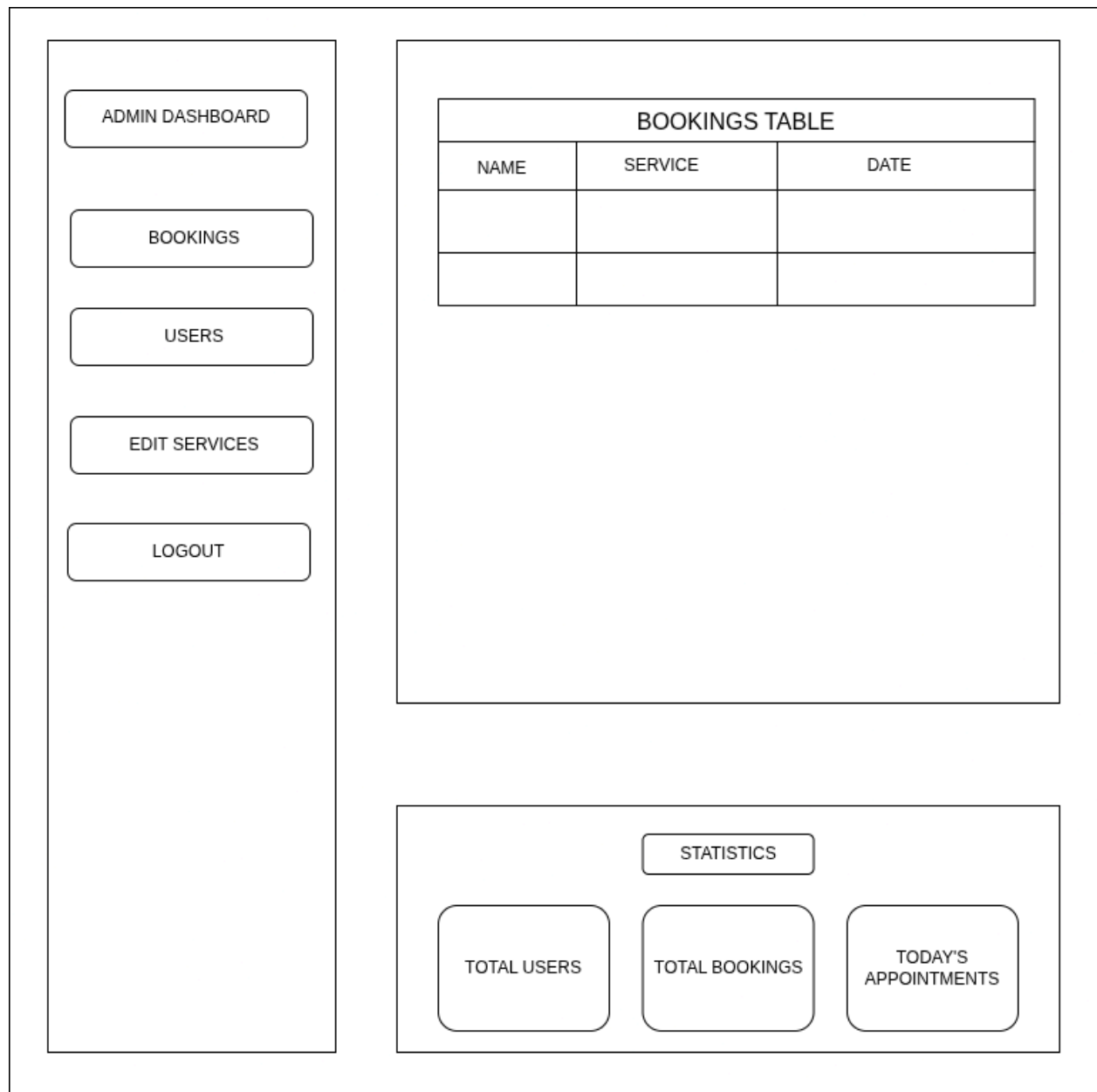
A user interface mockup for a login page. It features a central container with a white background and a thin black border. At the top, the text "WELCOME TO RUNI EMPIREE" is centered in a bold, black, sans-serif font. Below this, the label "Username:" is positioned to the left of a horizontal rectangular input field with rounded corners and a thin black border. Directly beneath the username field, the label "Password:" is positioned to the left of another identical horizontal rectangular input field. At the bottom of the container, a wide rectangular button with rounded corners and a thin black border is centered, containing the text "LOGIN" in a bold, black, sans-serif font.

WELCOME TO RUNI EMPIREE

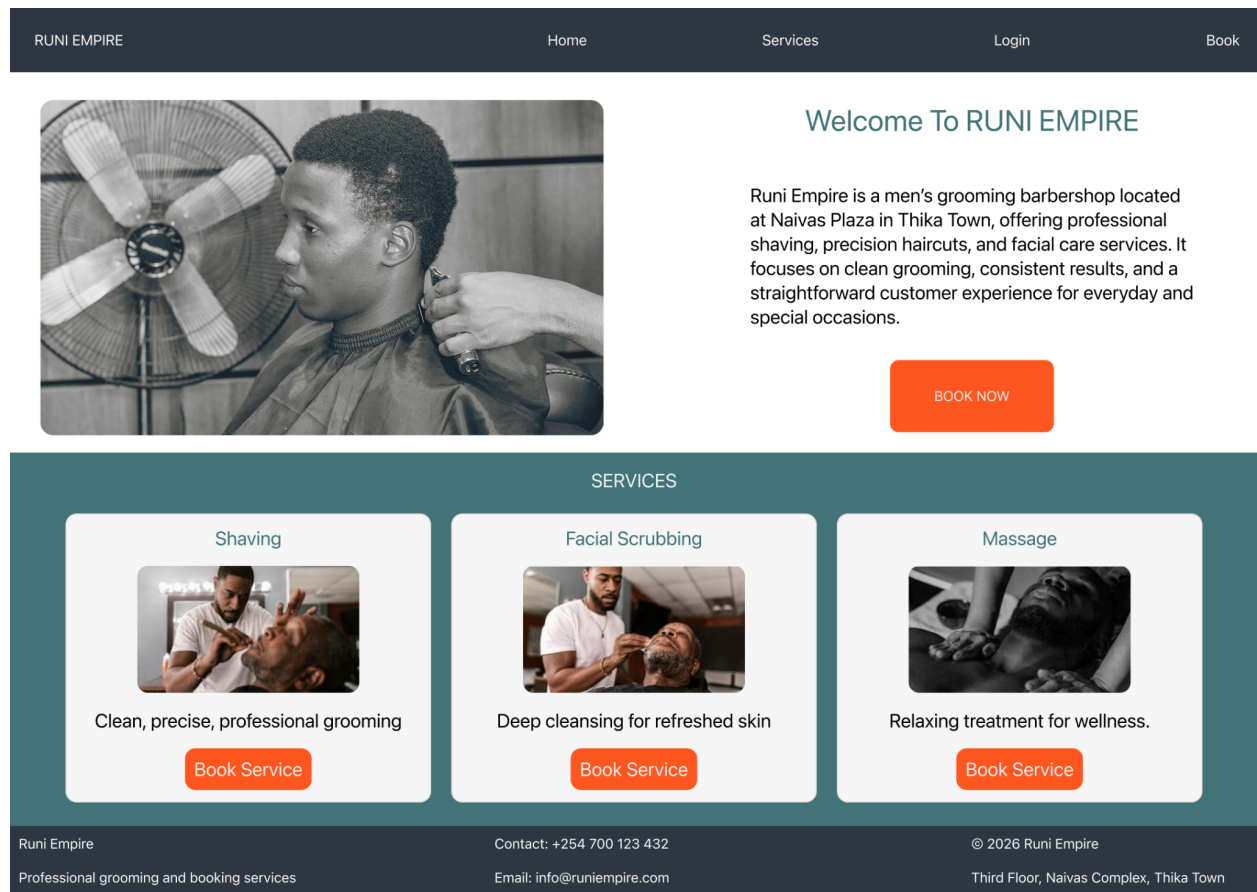
Username:

Password:

LOGIN

2.3. Figure 2.3: Admin Dashboard User Interface Mockup

2.4. Figure 2.4: Homepage Figma prototype



2.5. Figure 2.5: Login page Figma prototype

RUNI EMPIRE

Home

Services

Login

Book

Login to Access Our Services

Email:

Password:

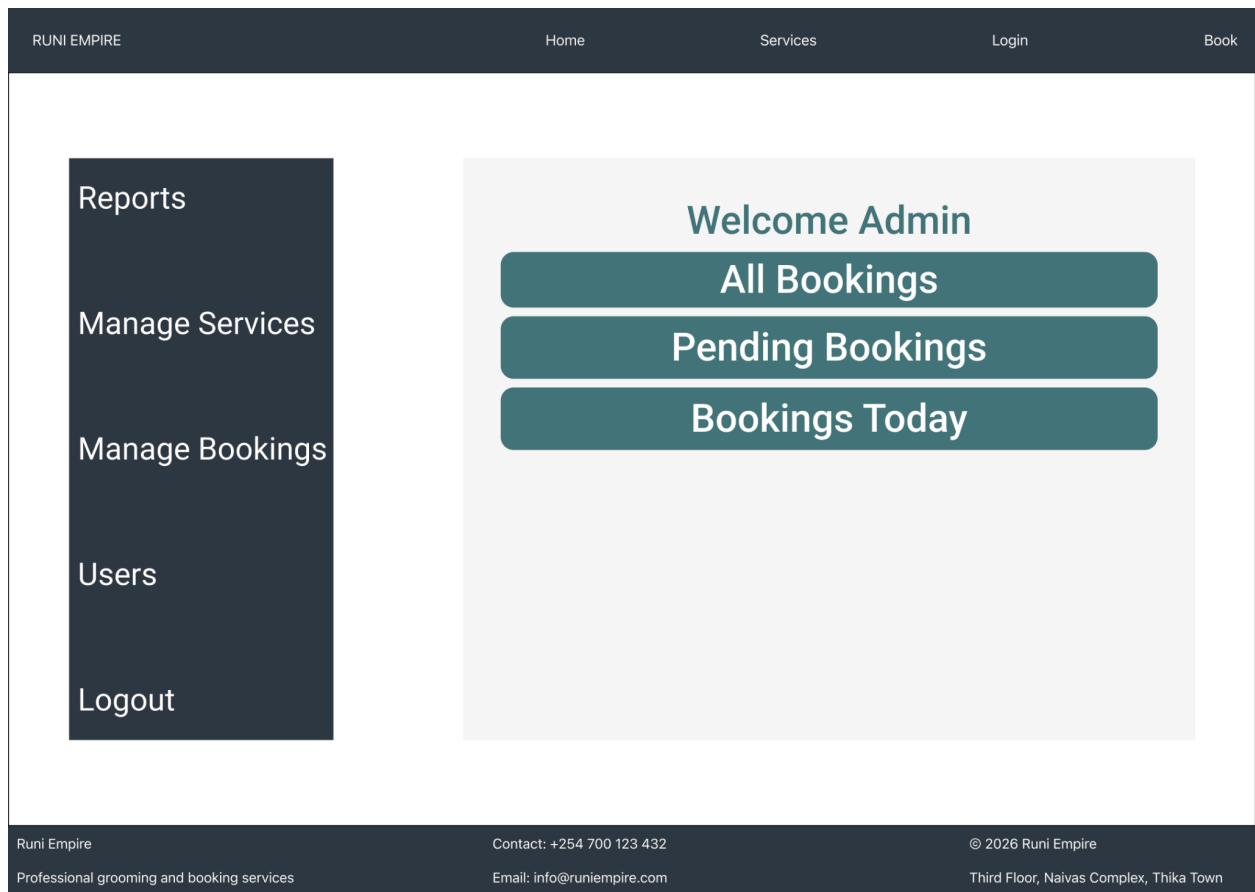
Login

Don't have an account?[Sign Up](#)

Runi Empire
Professional grooming and booking services

Contact: +254 700 123 432
Email: info@runiempire.com

© 2026 Runi Empire
Third Floor, Naivas Complex, Thika Town

2.6. Figure 2.6: Dashboard Figma prototype

2.7. Figure 2.7: Color Scheme

- i. Background: #F5F5F5 : Light Gray
- ii. Secondary/Highlights: #76ABAE: Teal
- iii. Main text: #303841: Charcoal
- iv. Accent/Actions: #FF5722: Orange

2.8. Figure 2.8: Navigation Structure

Reflection week 2:

Week 2 provided me with another key challenge in that after visualizing how my website should be laid out, the next obstacle was designing the prototype using Figma. To complete the process successfully, I had to consider carefully how to come up with reusable components to avoid creating frames repetitively.

3. Week 3: JavaScript and PHP Basics

3.1. Figure 1: Invalid Login Validation

Create Account to Access our Services

Name

Email

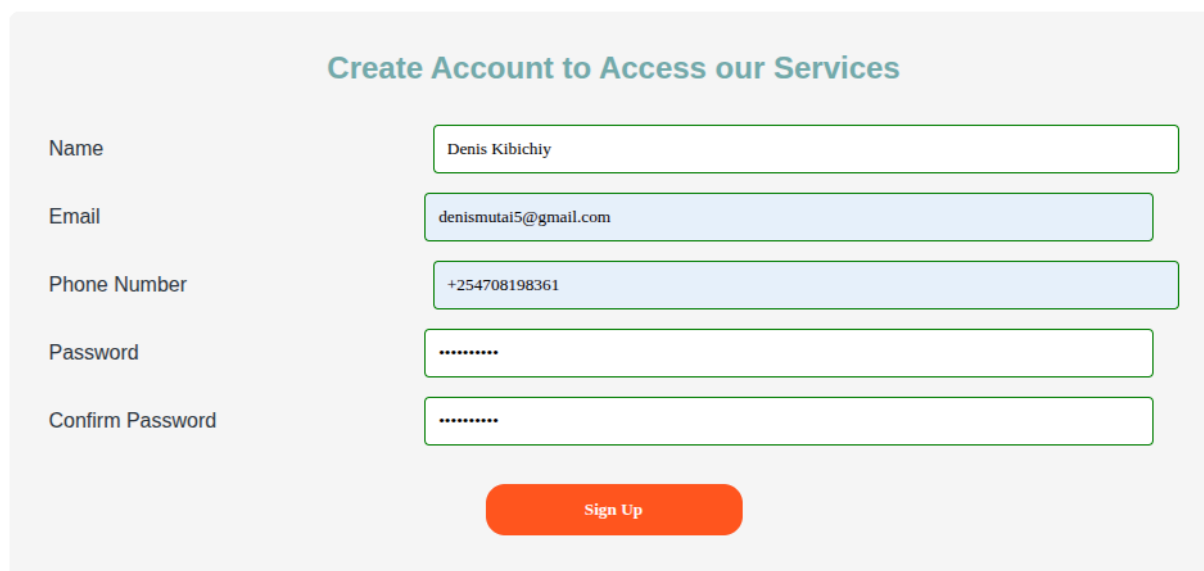
Phone Number

Password

Confirm Password Passwords must match

Already have an account? [Login](#)

3.2.Fig 2: Valid login input accepted

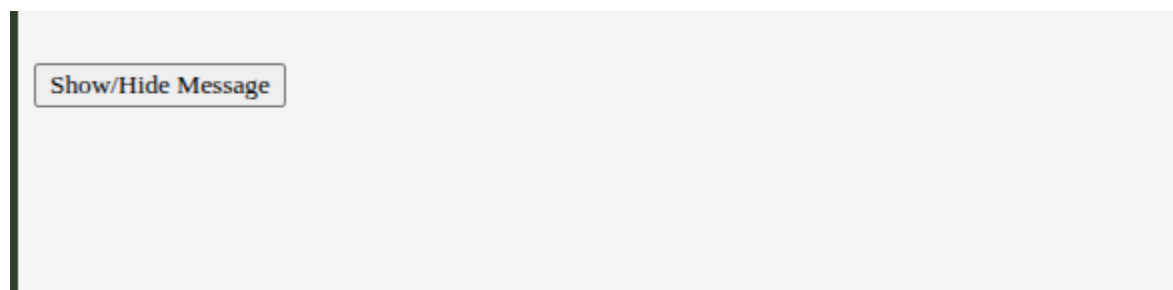


The screenshot shows a 'Create Account to Access our Services' form. The form fields and their values are:

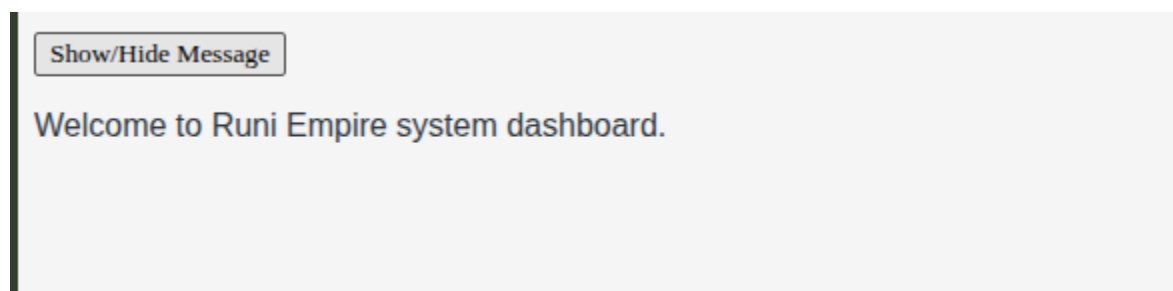
Field	Value
Name	Denis Kibichiy
Email	denismutai5@gmail.com
Phone Number	+254708198361
Password	*****
Confirm Password	*****

Below the fields is an orange 'Sign Up' button.

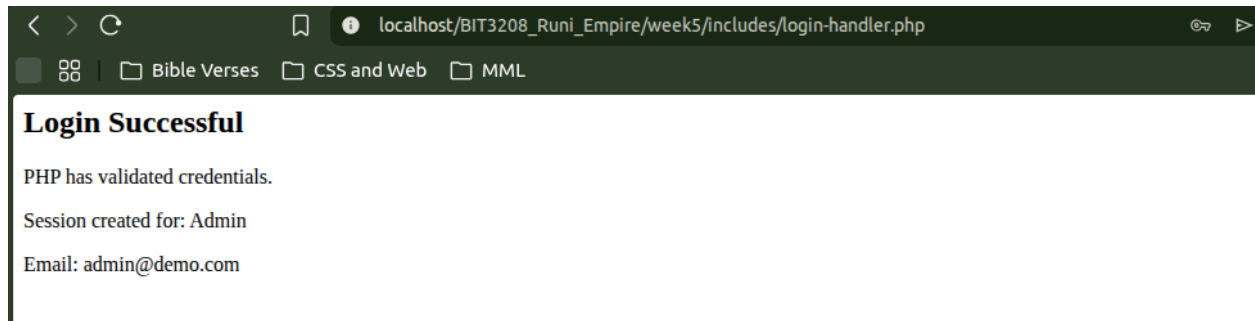
3.3.Fig 3: DOM manipulation interaction



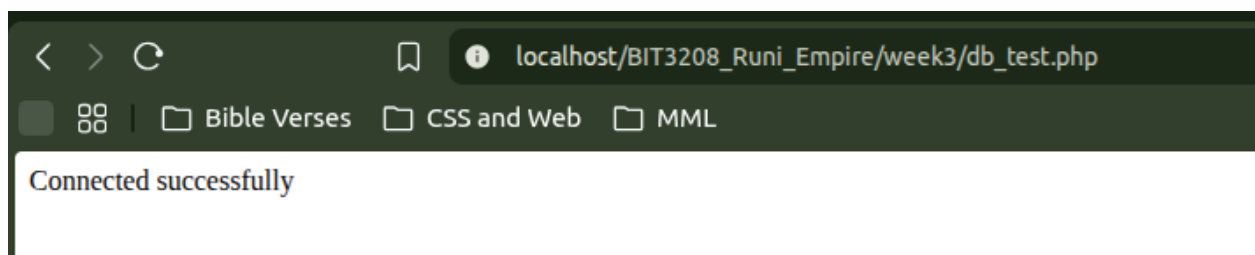
After button click



3.4. Figure 4: PHP Interaction Evidence after valid login details captured



3.5.Figure 5: Week 3 database connection success evidence

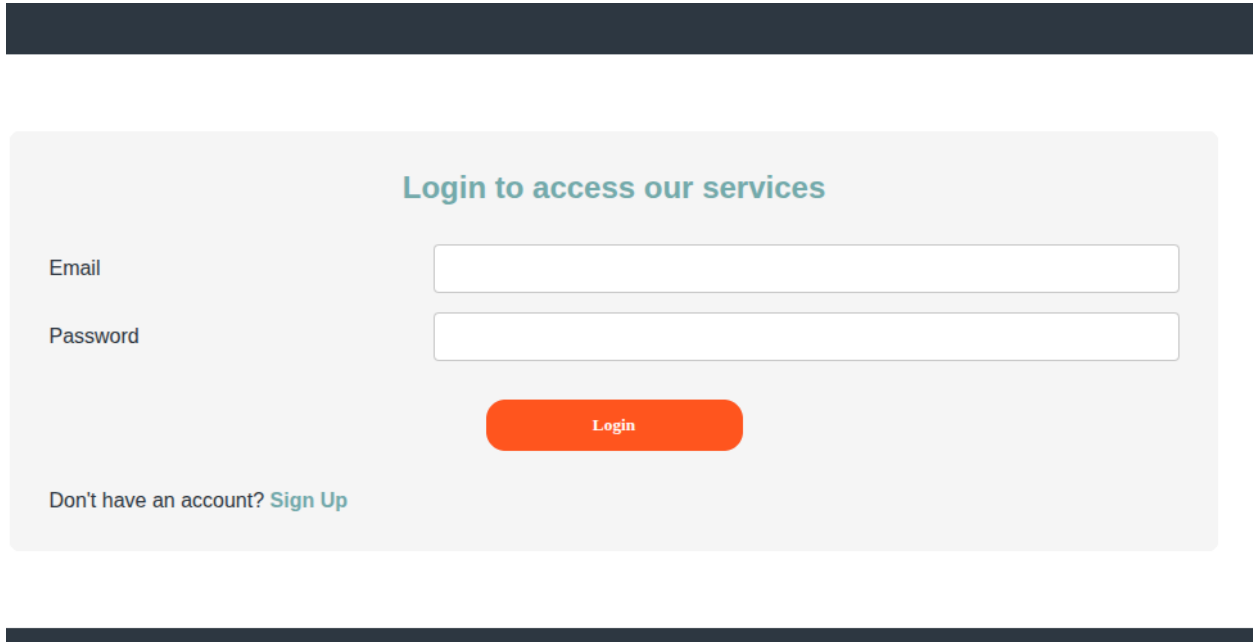


Week 3 Reflection:

In week 3, much of the focus shifted to JavaScript and PHP processing. The most challenging aspect was using JavaScript to perform form validation, and to succeed, I had to learn a lot of concepts, mostly dealing with DOM Manipulation such as query selectors, creating nodes, and appending them to already existing parent containers.

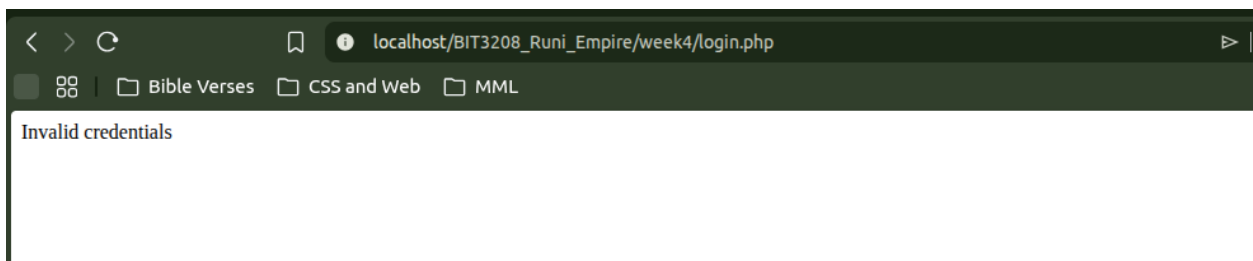
4. WEEK 4: Server-Side Components and Backend Development Foundations

4.1. Figure 1: Login page initial state



The image shows a login page with a dark blue header. Below the header is a light gray box containing the login form. The form has the title "Login to access our services" in teal. It includes two input fields: "Email" and "Password". Below the "Password" field is an orange "Login" button. At the bottom of the form, there is a link: "Don't have an account? [Sign Up](#)".

4.2. Figure 2: Failed Login Attempt



4.3. Figure 3: Successful login attempt - Redirected to dashboard

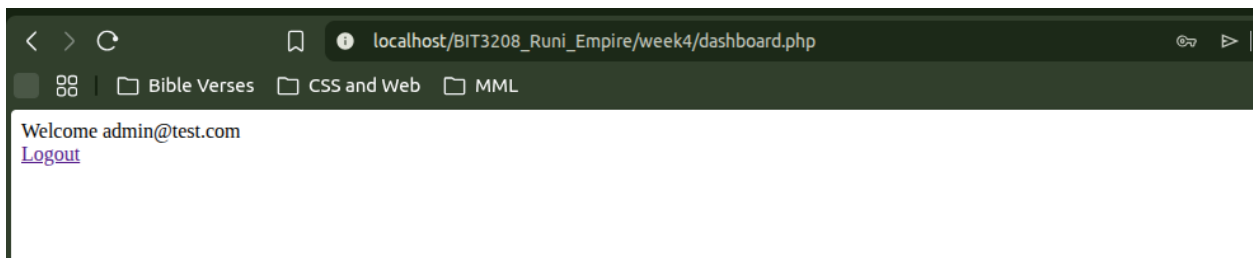
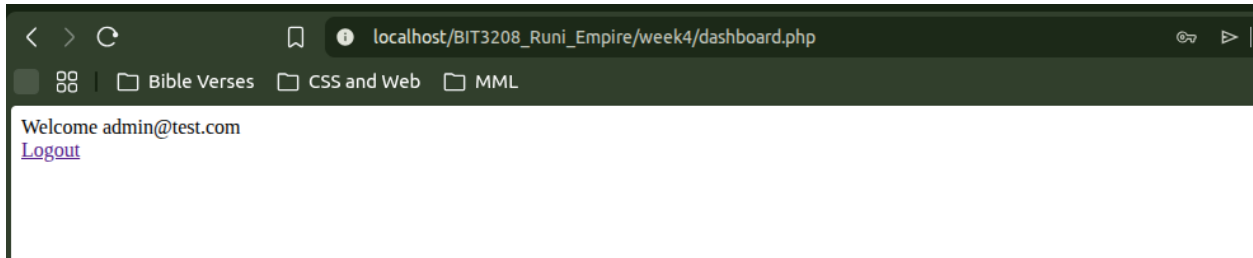
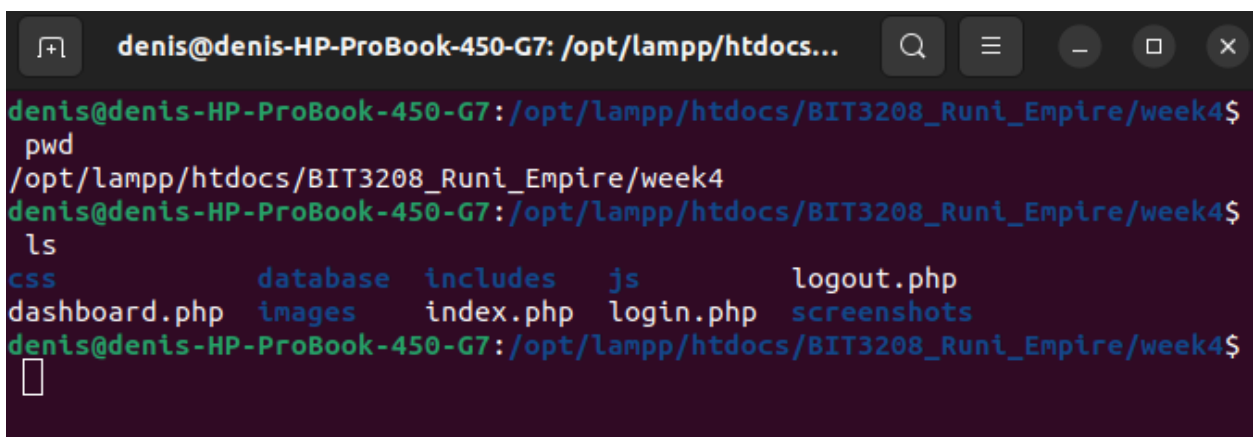


Figure 4: Session-based admin dashboard



4.4. Figure 5: Folder Structure



Week 4 Reflection:

In Week 4, the focus was on form processing, authentication basics, and sessions in PHP. The most difficult part was understanding how data moves from HTML forms into PHP and how validation is applied before processing. I learned how to capture form input using POST, validate it, and manage user sessions to maintain login state. I also built a basic login system and session-protected dashboard. This helped me understand how authentication systems control access to different parts of a web application.

5. Week 5: Database Components and CRUD Operations

5.1. Figure 5.1: Databases Screenshot

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| kinyozi_db |
| mysql |
| performance_schema |
| runi_empire_db |
| school |
| sys |
| week1db |
| week_one |
+-----+
9 rows in set (0.01 sec)
```

5.2. Figure 5.2: Created Tables within Database and defined their schema

```
mysql> USE runi_empire_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_runi_empire_db |
+-----+
| bookings |
| users |
+-----+
2 rows in set (0.00 sec)

mysql> DESCRIBE users;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id | int | NO | PRI | NULL | auto_increment |
| first_name | varchar(100) | YES | | NULL | |
| last_name | varchar(100) | YES | | NULL | |
| email | varchar(100) | YES | UNI | NULL | |
| phone_number | varchar(20) | YES | | NULL | |
| password | varchar(255) | YES | | NULL | |
| created_at | timestamp | YES | | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+
7 rows in set (0.00 sec)
```

5.3. Figure 5.3: CRUD Operations on database

```
mysql> INSERT INTO users (first_name, last_name, email, phone_number, password)
-> VALUES ('Brian', 'Kiptoo', 'brian.kiptoo@gmail.com', '0712345678', '1234');
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM users;
+-----+-----+-----+-----+-----+-----+-----+
| id | first_name | last_name | email | phone_number | password | created_at |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | Brian | Kiptoo | brian.kiptoo@gmail.com | 0712345678 | 1234 | 2026-06-05 11:47:07 |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> UPDATE users
-> SET first_name = 'Kevin',
-> last_name = 'Mutua'
-> WHERE id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT * FROM users;
+-----+-----+-----+-----+-----+-----+-----+
| id | first_name | last_name | email | phone_number | password | created_at |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | Kevin | Mutua | brian.kiptoo@gmail.com | 0712345678 | 1234 | 2026-06-05 11:47:07 |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> DELETE FROM users
-> WHERE id = 1;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM users;
Empty set (0.00 sec)

mysql>
```

Week 5 Reflection:

In Week 5, I focused on databases and integrating them with PHP applications. The main challenge was understanding SQL operations and how PHP interacts with MySQL databases. I learned how to create databases and tables, perform CRUD operations, and connect PHP to a database using PDO. I also implemented database-driven login functionality and data retrieval. This improved my understanding of backend development and data management in web applications.

6. Week 6: Database Integration and CRUD Operations

6.1. Database Creation (Already done in week 5 but now normalized)

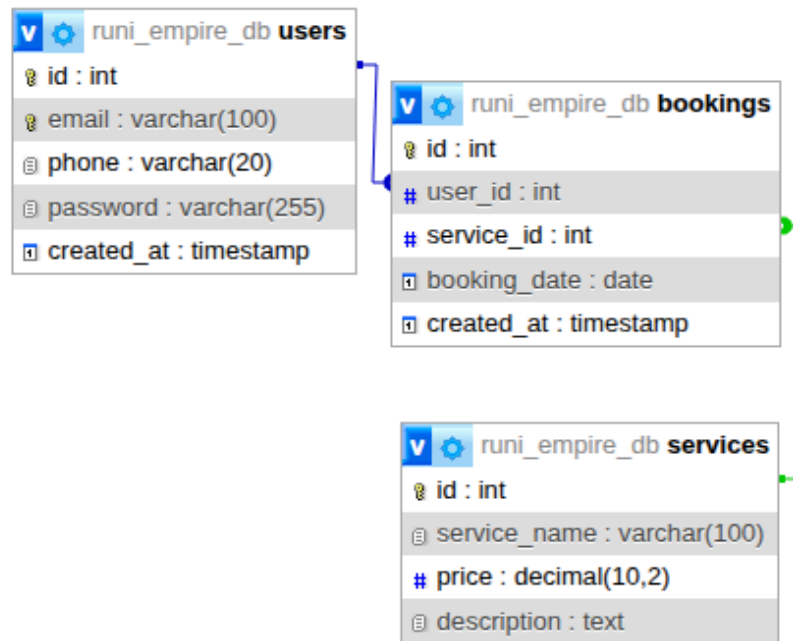
Users table normalized

```
mysql> DESCRIBE users;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default          | Extra          |
+-----+-----+-----+-----+-----+-----+
| id    | int           | NO   | PRI | NULL             | auto_increm   |
| email | varchar(100)  | YES  | UNI | NULL             |               |
| phone | varchar(20)   | YES  |     | NULL             |               |
| password | varchar(255) | YES  |     | NULL             |               |
| created_at | timestamp    | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.01 sec)
```

New Bookings Table Introduced Using FK

```
mysql> CREATE TABLE bookings (
  ->   id INT AUTO_INCREMENT PRIMARY KEY,
  ->   user_id INT NOT NULL,
  ->   service_id INT NOT NULL,
  ->   booking_date DATE NOT NULL,
  ->   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  ->
  ->   CONSTRAINT fk_booking_user
  ->     FOREIGN KEY (user_id)
  ->     REFERENCES users(id),
  ->
  ->   CONSTRAINT fk_booking_service
  ->     FOREIGN KEY (service_id)
  ->     REFERENCES services(id)
  -> );
Query OK, 0 rows affected (0.06 sec)
```

6.2. Entity Relationship Diagram



6.3. User Registration Form on Submit submits to the Users table

User Inputting the details

Create Account to Access our Services

Name

Georgina Thuo

Email

georginathuo@gmail.com

Phone Number

0712345678

Password

.....

Confirm Password

.....

Sign Up

User appearing on the table

```
mysql> SELECT * FROM users WHERE email = 'georginathuo@gmail.com';
+----+-----+-----+-----+-----+-----+
| id | name          | email                | phone      | password    |
|    |               |                      | created_at |              |
+----+-----+-----+-----+-----+-----+
| 7  | Georgina Thuo | georginathuo@gmail.com | 0712345678 | $2y$10$V871xLyLYPF.JHWBg9QDSeCPVLW91DY1MVwSwlksL5sMx6NVcmHzu | 2026-06-18 11:49:45 |
+----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

6.4. Update and Delete Records via the terminal evidence

```
mysql> UPDATE users
-> SET name = 'Brian Kiptoo'
-> WHERE email = 'brian.kiptoo@gmail.com';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE users
-> SET name = 'Faith Wanjiru'
-> WHERE email = 'faith.wanjiru@gmail.com';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT id, name, email, phone FROM users;
+----+-----+-----+-----+
| id | name          | email                | phone      |
+----+-----+-----+-----+
| 2  | Brian Kiptoo  | brian.kiptoo@gmail.com | 0712345678 |
| 3  | Faith Wanjiru | faith.wanjiru@gmail.com | 0722113344 |
| 4  | NULL         | kevin.mutua@gmail.com  | 0700123456 |
| 5  | NULL         | amina.mohamed@gmail.com | 0799887766 |
| 6  | NULL         | dennis.otieno@gmail.com | 0733557799 |
| 7  | Georgina Thuo | georginathuo@gmail.com | 0712345678 |
+----+-----+-----+-----+
6 rows in set (0.00 sec)
```

Delete

```
mysql> DELETE FROM users
      -> WHERE email != 'georginathuo@gmail.com';
Query OK, 5 rows affected (0.02 sec)

mysql> SELECT * FROM users;
+----+-----+-----+-----+-----+-----+
| id | name          | email                | phone      | password |
|    |              |                    | created_at |          |
+----+-----+-----+-----+-----+-----+
|  7 | Georgina Thuo | georginathuo@gmail.com | 0712345678 | $2y$10$V871xLyLYPF.JHWBg9QDSeCPVLW91DY1MVwSwlksL5sMx6NVcmHzu |
|    |              |                    | 2026-06-18 11:49:45 |          |
+----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> █
```

Week 6 reflection:

In README

7. Week 7: User Authentication and Session Management

7.1. Figure 7.1: users table with different roles included

```
mysql> DESCRIBE users;
+-----+-----+-----+-----+-----+-----+
| Field | Type                | Null | Key | Default          | Extra          |
+-----+-----+-----+-----+-----+-----+
| id    | int                 | NO   | PRI | NULL             | auto_increment |
| name  | varchar(100)        | YES  |     | NULL             |                |
| email | varchar(100)        | YES  | UNI | NULL             |                |
| phone | varchar(20)         | YES  |     | NULL             |                |
| password | varchar(255)      | YES  |     | NULL             |                |
| created_at | timestamp        | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| role  | enum('user','admin','super_admin') | YES  |     | user             |                |
+-----+-----+-----+-----+-----+-----+
7 rows in set (0.00 sec)
```

7.2. Figure 7.2: Role assignments done successfully

```
mysql> SELECT id, name, email, role FROM users;
+-----+-----+-----+-----+
| id | name       | email                | role    |
+-----+-----+-----+-----+
| 1 | Georgina Thuo | georginathuo@gmail.com | super_admin |
| 2 | Denis Kibichiy | denismutai5@gmail.com | admin      |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

7.3. Figure7.3: Normal User Registration

Create Account to Access our Services

Name

Email

Phone Number

Password

Confirm Password

[Sign Up](#)

Already have an account? [Login](#)

7.4. Figure 7.4: Normal User saved with role successfully

```
mysql> SELECT id,name,email,role
-> FROM users;
+-----+-----+-----+-----+
| id | name       | email                | role    |
+-----+-----+-----+-----+
| 1 | Georgina Thuo | georginathuo@gmail.com | super_admin |
| 2 | Denis Kibichiy | denismutai5@gmail.com | admin      |
| 3 | Elvis Kipkosgei | elvis1234@gmail.com | user      |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

7.5. Figure 7.5: Password hashing for security purposes

```
mysql> SELECT email,password
-> FROM users;
+-----+-----+
| email                | password |
+-----+-----+
| georginathuo@gmail.com | $2y$10$BCKp5kUA47Lt4wPZlPizY04.0yA5SgyxCeYTr3.u4jpSYoz2NRXsu |
| denismutai5@gmail.com  | $2y$10$XH9wa0zzp1U.1XzD80Xrm.xQqkD/7lVAy64aHtBccZWWhQZQsfTVS |
| elvis1234@gmail.com    | $2y$10$P7CWHU3V2Nr2FxYrUSua4eFh3l9coEbyj/w2zmMMz25RC0oKbUyWe |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

7.6. Figure 7.6: Super Admin Dashboard with super admin privileges extends 2 admin privileges (creating users and bookings)

RUNI EMPIRE
Home
Login
Book
Services

Welcome Super Admin
Role: super_admin
Super Admin Dashboard
My Profile
Manage Users
Manage Admins
Manage Services
View Bookings
Logout

7.6.1. Super admin can edit admins

RUNI EMPIRE
Home
Login
Book
Services

Manage Admins
Create Admin

ID	Name	Email	Actions
2	Denis Kibichiy	denismutai5@gmail.com	Delete
6	New Admin	newadmin@gmail.com	Delete

Back

7.6.2. Super admin can create services

RUNI EMPIRE	Home	Login	Book	Services
-------------	------	-------	------	----------

Add Service

Service Name

Price

Description

[Back](#)

7.7. Admin dashboard showing admin privileges that can also be done by super admin

RUNI EMPIRE	Home	Login	Book	Services
-------------	------	-------	------	----------

Welcome Denis Kibichiy

Role: admin

Admin Dashboard

[My Profile](#)
[View Users](#)
[View Bookings](#)
[Logout](#)

7.7.1. Admin can create users (admin + super admin)

RUNI EMPIRE	Home	Login	Book	Services
-------------	------	-------	------	----------

Create User

Name Email Phone Password

[Back](#)

7.7.2. Admin can create bookings that can also be done by the super admin

RUNI EMPIRE
Home
Login
Book
Services

Create Booking

Super Admin
Shaving
mm/dd/yyyy
Create Booking

[Create User](#)
[Back](#)

7.7.3. Admin can't edit fellow admins or super admin

RUNI EMPIRE
Home
Login
Book
Services

Registered Users

ID	Name	Email	Phone	Created At	Role	Actions
1	Super Admin	georginathuo@gmail.com	0712345678	2026-06-25 11:14:26	super_admin	Not allowed
2	Denis Kibichiy	denismutai5@gmail.com	0712345687	2026-06-25 11:19:45	admin	Not allowed
4	Ken Kiprotich	kenkiprotich@gmail.com	0712345678	2026-06-25 14:46:09	user	Edit Delete
8	New Admin	newadmin@gmail.com	0712345678	2026-06-26 07:27:51	admin	Not allowed

[Create User](#)
[Back](#)

7.8. Normal user profile with limited privileges: edit profile, create booking and cancel booking

7.8.1. Normal user can view their bookings and add or cancel

RUNI EMPIRE
Home
Login
Book
Services

My Bookings

ID	Service	Booking Date	Created At	Action
4	Massage	2026-06-25	2026-06-26 08:08:09	Cancel

[Back to Dashboard](#)

7.8.2. *A normal user can add a booking*

RUNI EMPIRE

HomeLoginBookServices

Service

Shaving

Booking Date

mm / dd / yyyy

Book

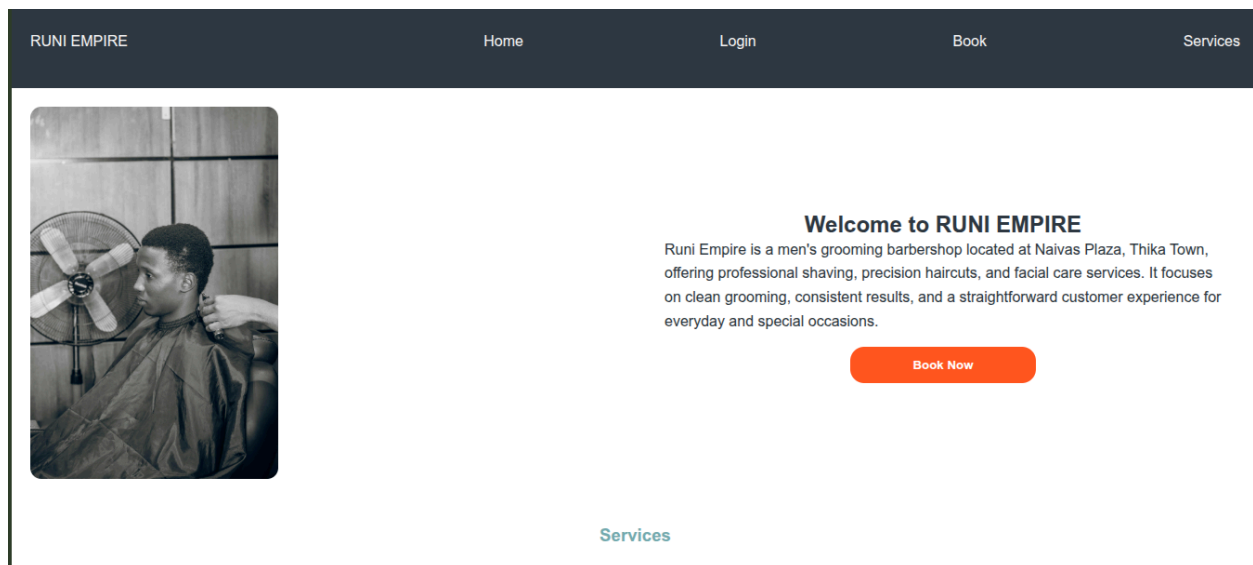
Back

7.9. Week 7 reflection

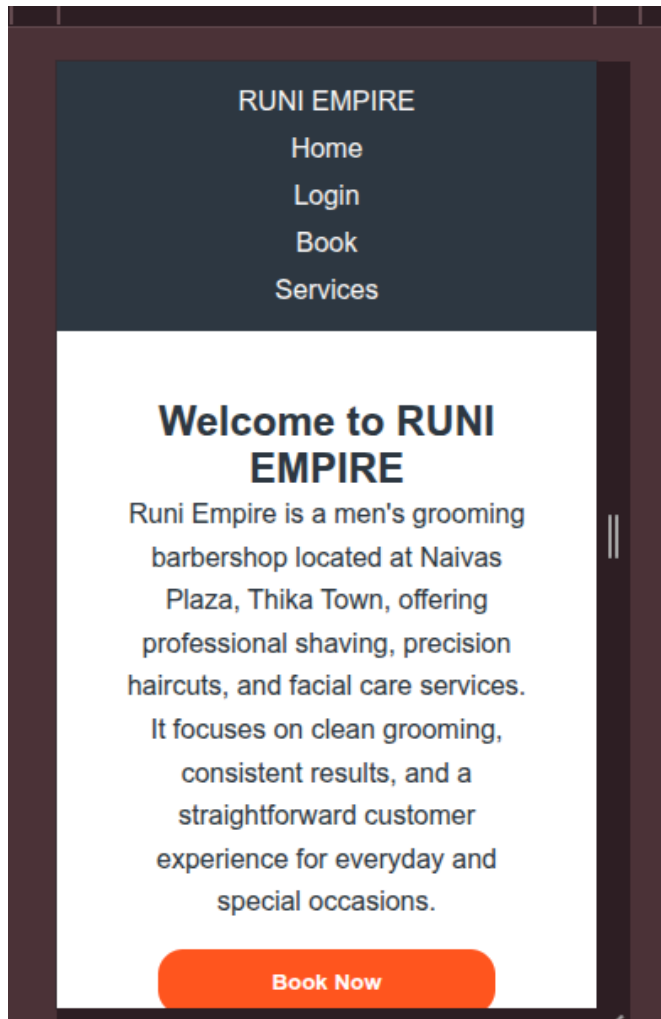
In README (root README)

8. Week 8: Responsive Design

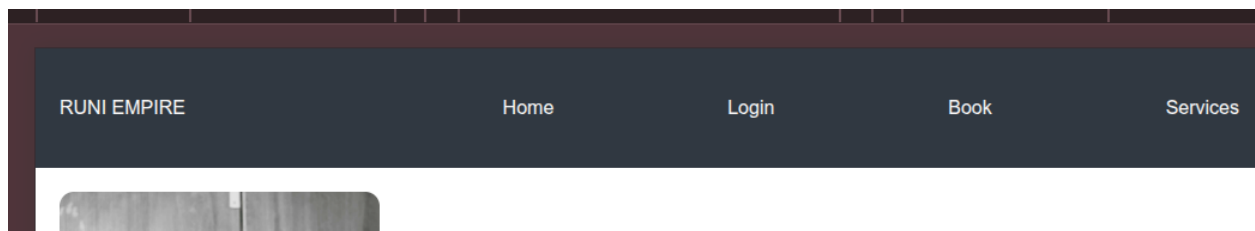
8.1. Figure 8.1: Home page desktop view



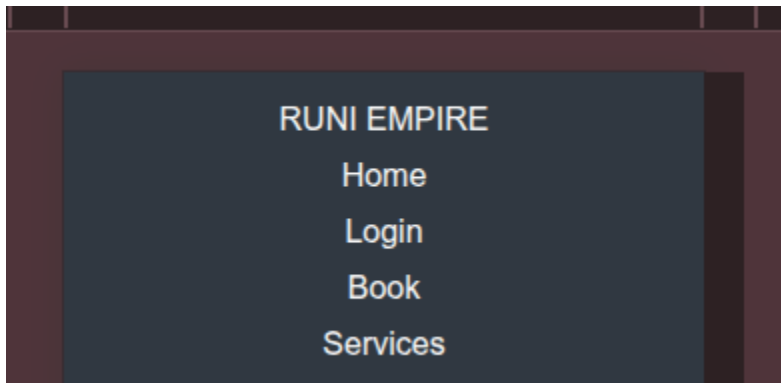
8.2. Figure 8.2: Home page mobile view



8.3. Figure 8.3: Navigation Desktop view



8.4. Figure 8.4: Navigation Mobile view



8.5. Figure 8.5: Responsive tables using horizontal scroll

RUNI EMPIRE					
Home					
Login					
Book					
Services					
Registered Users					
ID	Name	Email	Phone	Created At	Ro
1	Super Admin	georginathuo@gmail.com	0712345678	2026-06-25 11:14:26	super_
2	Denis Kibichiy	denismutai5@gmail.com	0712345687	2026-06-25 11:19:45	adm
4	Ken Kiprotich	kenkiprotich@gmail.com	0712345678	2026-06-25 14:46:09	usu
8	New Admin	newadmin@gmail.com	0712345678	2026-06-26 07:27:51	adm

[Create User](#)
[Back](#)

8.6. Week 8 reflection

9. Week 9: Java Web Services: Servlet Lifecycle, Session Management, and Cookies

9.1. Figure 9.1: Login page: all modules protected from invalid sessions

Home Register Patient Patients Admissions

Preferred Theme: Light ▾

EHealth System Login

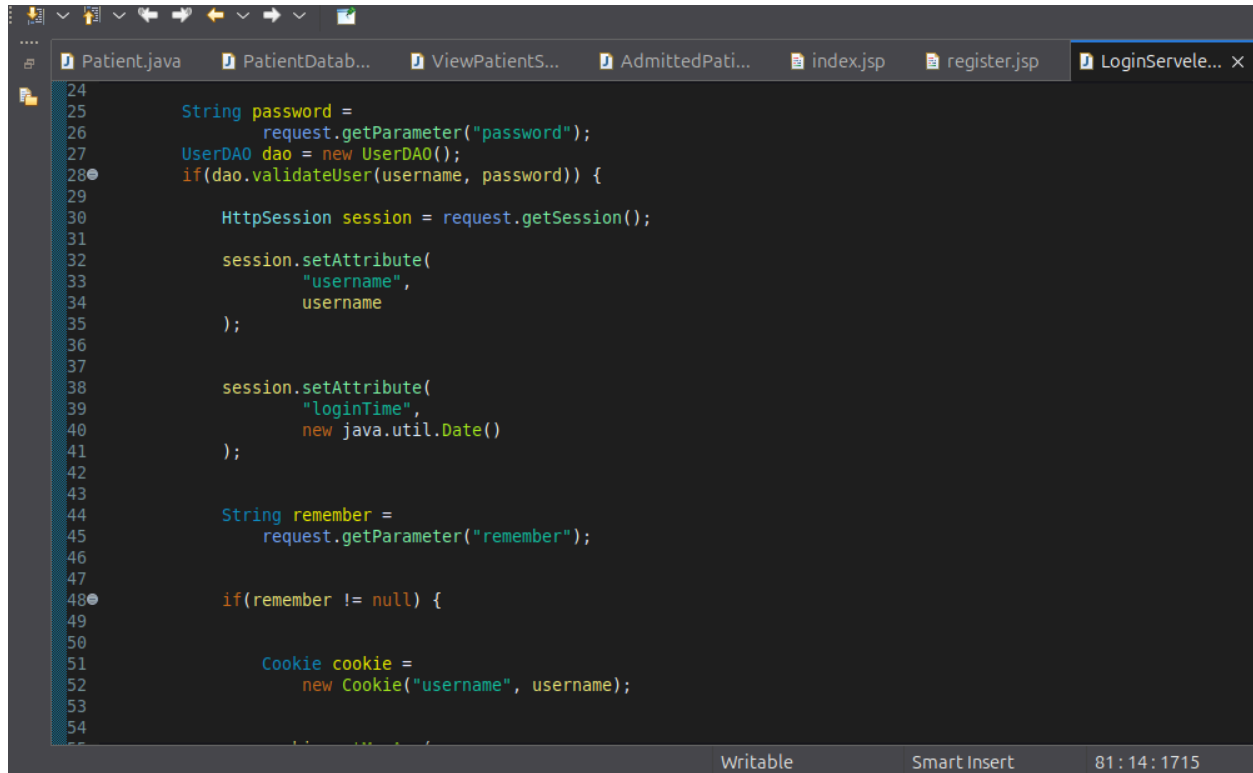
Username

Password

☐ Remember Me

Login

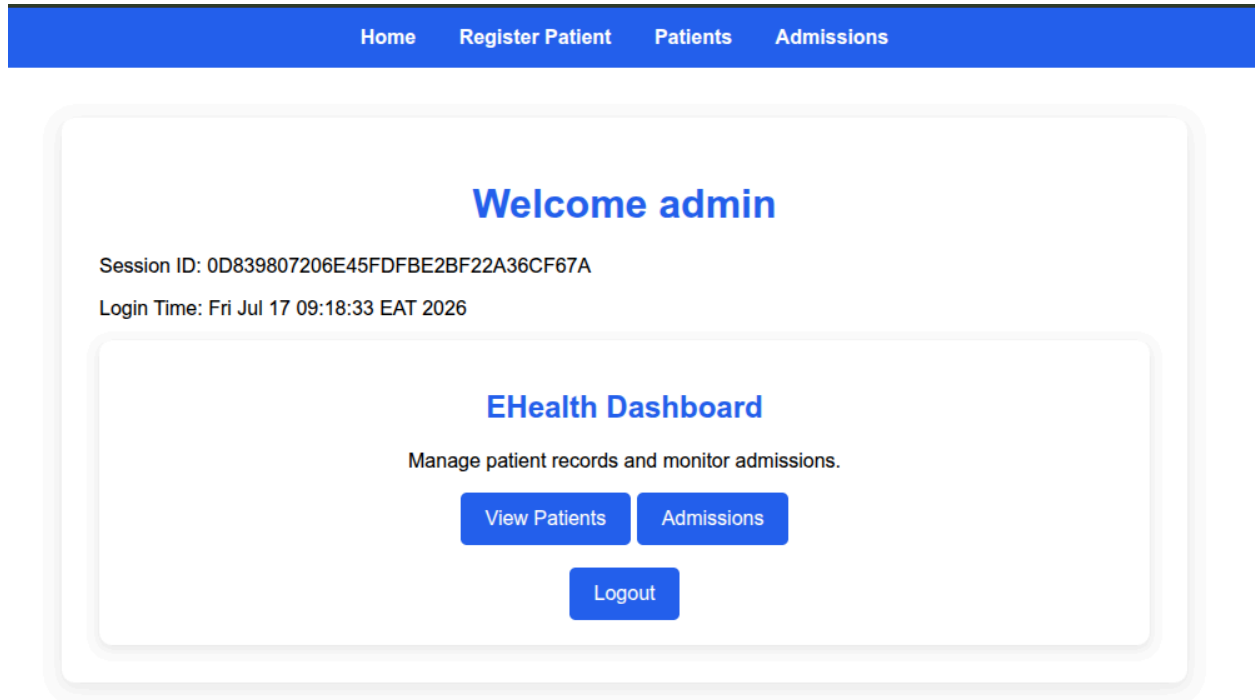
9.2. Figure 9.2: LoginServlet code showing credential validation and session creation



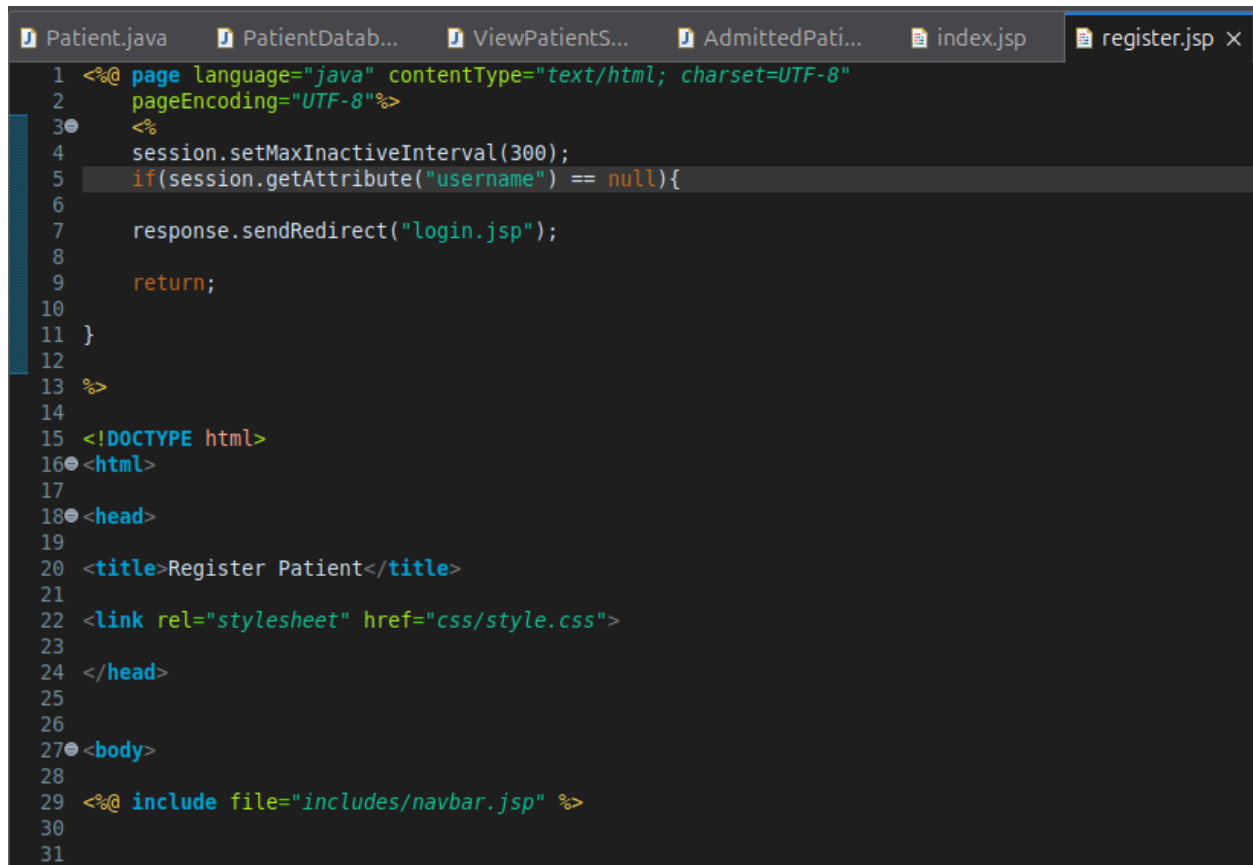
```
24
25
26     String password =
27         request.getParameter("password");
28     UserDAO dao = new UserDAO();
29     if(dao.validateUser(username, password)) {
30
31         HttpSession session = request.getSession();
32
33         session.setAttribute(
34             "username",
35             username
36         );
37
38         session.setAttribute(
39             "loginTime",
40             new java.util.Date()
41         );
42
43
44         String remember =
45             request.getParameter("remember");
46
47         if(remember != null) {
48
49             Cookie cookie =
50                 new Cookie("username", username);
51
52
53
54
```

Writable Smart Insert 81 : 14 : 1715

9.3. Figure 9.3: Successful login redirecting to dashboard

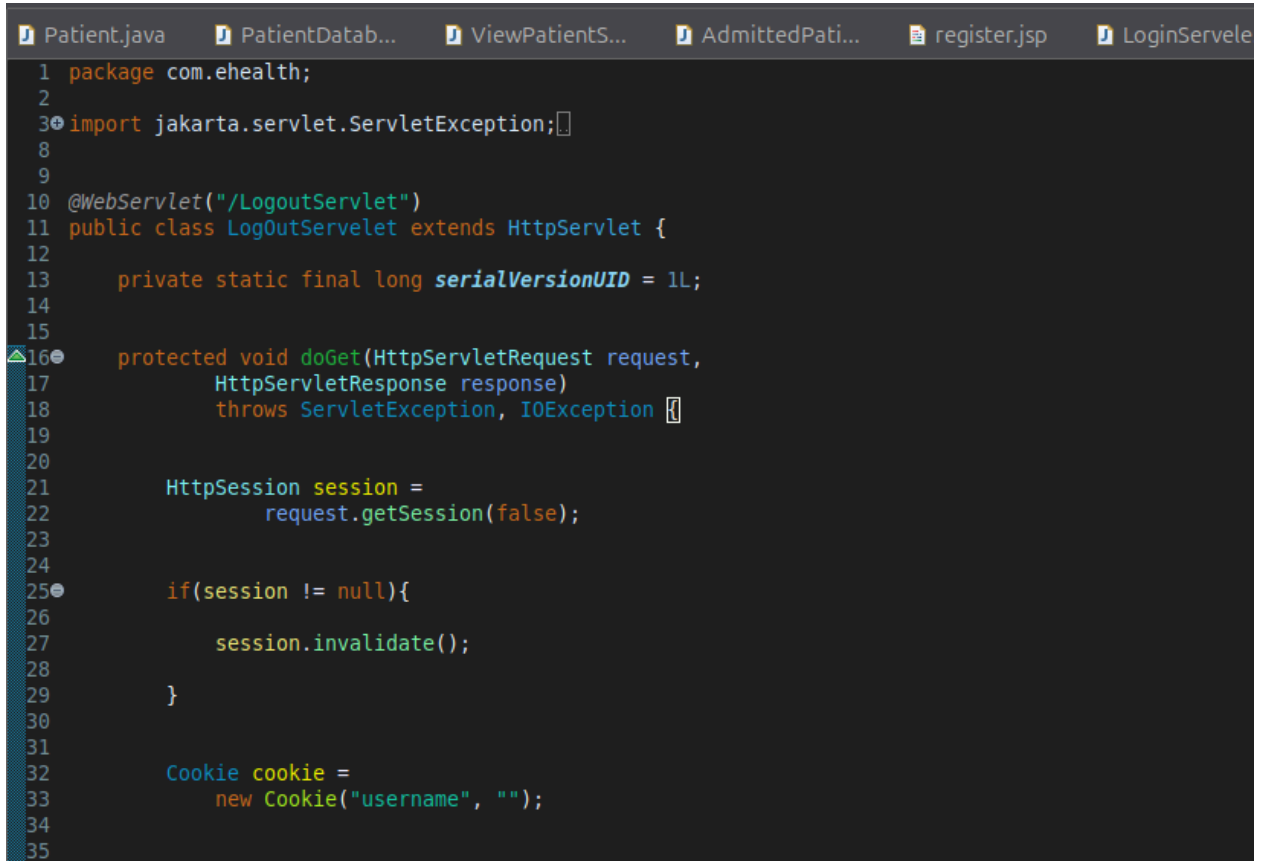


9.4. Figure 9.4: Modules protected from access if user is not logged in



```
1 <%@ page language="java" contentType="text/html; charset=UTF-8"
2   pageEncoding="UTF-8"%>
3 <%
4   session.setMaxInactiveInterval(300);
5   if(session.getAttribute("username") == null){
6
7     response.sendRedirect("login.jsp");
8
9     return;
10  }
11 }
12 %>
13 <!DOCTYPE html>
14 <html>
15 <head>
16 <title>Register Patient</title>
17 <link rel="stylesheet" href="css/style.css">
18 </head>
19 <body>
20 <%@ include file="includes/navbar.jsp" %>
21
22
23
24
25
26
27
28
29
30
31
```

9.5. Figure 9.5: Logout functionality implemented to destroy the session if the button is clicked



```

1 package com.ehealth;
2
3 import jakarta.servlet.ServletException;
4
5
6
7
8
9
10 @WebServlet("/LogoutServlet")
11 public class LogoutServlet extends HttpServlet {
12
13     private static final long serialVersionUID = 1L;
14
15
16     protected void doGet(HttpServletRequest request,
17         HttpServletResponse response)
18         throws ServletException, IOException {
19
20
21         HttpSession session =
22             request.getSession(false);
23
24
25         if(session != null){
26             session.invalidate();
27         }
28
29
30
31         Cookie cookie =
32             new Cookie("username", "");
33
34
35

```

9.6. Week 9 Reflection

Week 9 focused on implementing authentication, session tracking, and cookie management in the EHealth Patient Management System. I learned how Java Servlets and JSP pages work together to authenticate users and control access to protected resources. The implementation of HttpSession helped me understand how user information can be maintained across multiple requests, while cookies demonstrated how preferences and login-related data can be stored. Challenges included configuring Tomcat, debugging session protection, and ensuring proper redirection after logout. Overall, the practical improved my understanding of building secure Java web applications with authentication mechanisms.

10. Week 10: